

MATRIX Remote QM Execution on oracle

Running Gaussian, Molpro and ORCA jobs for MATRIX workflows

MATRIX Development Notes

June 28, 2026

Abstract

This manual describes the operational contract for running external quantum chemistry programs on the private Linux host **oracle** and bringing the completed results back into **MATRIX**. The remote machine is used only for execution of heavy QM jobs. **MATRIX** remains the owner of the molecular state, shared **xyzin** sections, generated Gaussian GIC inputs and normalized post-processing through the single QM-adapter layer.

Contents

1	Scope	2
2	Remote Directory Layout	2
3	Environment	2
4	Direct Program Commands	3
4.1	Gaussian GDV32	3
4.2	Gaussian G16	3
4.3	Molpro	3
4.4	ORCA	4
5	Background Submission	4
6	Copying Files	4
7	Returning Results To MATRIX	5
8	Troubleshooting	5
9	Operational Policy	6

1 Scope

The **oracle** machine is intended for long background calculations with Gaussian, Molpro and ORCA. It is not a replacement for the local **MATRIX** source tree and it must not become a second set of parsers. The normal workflow is:

1. prepare or validate the molecule locally with **MATRIX**;
2. generate the QM input locally when **MATRIX** owns the input format, for example Gaussian ReadAllGIC files from frozen NEO/GICForge data;
3. copy the input file to **oracle**;
4. submit the job in the remote `~/matrix` work area;
5. copy completed logs, checkpoint-derived files or Molden files back;
6. promote the output through the corresponding **MATRIX** adapter into shared **xyzin** sections.

Passwords, personal credentials and site-specific secrets must not be written in the repository, in job inputs or in shell scripts. Use the interactive SSH prompt or a normal SSH key managed outside the repository.

2 Remote Directory Layout

The remote execution area is `~/matrix`. The following subdirectories have fixed meanings:

Directory	Purpose
<code>~/matrix/bin</code>	MATRIX helper scripts on the remote host.
<code>~/matrix/inputs</code>	Input files copied from the local workstation.
<code>~/matrix/jobs</code>	Per-job working directories created by the submit helper.
<code>~/matrix/logs</code>	Lightweight submit logs and redirected program output.
<code>~/matrix/outputs</code>	Optional collection area for selected final products.
<code>~/matrix/repo</code>	Optional clone or mirror of MATRIX when needed for tests.
<code>~/matrix/scratch</code>	MATRIX-owned scratch area; large Gaussian scratch is controlled separately.

Gaussian scratch is configured through **GAUSS_SCRDIR**. The current policy is to keep large Gaussian scratch under the local scratch filesystem configured on **oracle**, not inside the Git repository.

3 Environment

The login shell on **oracle** sources `~/matrix_env.sh` from both `~/bashrc` and, when available, `~/zshrc`. The environment file is intentionally narrow: it exposes only the production Gaussian choices and the supported module stack.

```
setgau gdv32
setgau g16
module load molpro/2025.3
module load orca/6.1.1
```

Older GDV variants are disabled for MATRIX use. Jobs must use only `gdv32` or `g16`. Do not source historical external-code environments such as `elecext-env`, `External` or `RunGauExternal`; they are outside the MATRIX execution contract.

Useful checks after login are:

```
command -v gdv
command -v g16
command -v molpro
command -v orca
module list
echo "$GAUSS_SCRDIR"
```

The expected Molpro version for this machine is Molpro 2025.3. If `module load molpro/2025.3` stops working, treat that as an environment configuration problem and do not silently fall back to an older Molpro module.

4 Direct Program Commands

The direct commands are useful for small checks and for debugging the environment.

4.1 Gaussian GDV32

```
setgau gdv32
gdv32run molecule.gjf
```

Use GDV32 for Gaussian jobs that need the current group-specific Gaussian features. For MATRIX-generated ReadAllGIC optimizations, the Gaussian route must use `Opt=ReadAllGIC`; for single-point jobs with a GIC geometry read, use the corresponding `Geom` form. MATRIX symmetrizes the GICs itself, so Gaussian-side automatic GIC symmetrization should not be requested.

4.2 Gaussian G16

```
setgau g16
g16run molecule.gjf
```

Use G16 when the job does not need GDV32-specific behavior. The same MATRIX adapter policy applies: logs, FCHK files and QFF data are copied back and then promoted through `matrix gaussian ...` commands.

4.3 Molpro

```
module load molpro/2025.3
molpro molecule.com
```

Molpro outputs enter MATRIX through the Molpro adapter. Downstream tools should consume the promoted `xyzin` sections and must not parse Molpro text privately.

4.4 ORCA

```
module load orca/6.1.1
orca molecule.inp > molecule.out
```

ORCA calculations can be launched on **oracle** for electronic-structure data and future MATRIX adapters. Until a given ORCA output quantity is normalized into a shared **xyzin** section, scientific MATRIX tools should not depend on tool-local ORCA parsing.

5 Background Submission

For production use, prefer the lightweight MATRIX submit helpers installed in `~/matrix/bin`. The source version of the submit helper is stored in the MATRIX repository as `scripts/remote/oracle/matrix-submit`. It creates one working directory per job, records the native program output, exposes a stable log link in `~/matrix/logs` and keeps a small metadata file with the command and process id.

```
matrix-submit gdv32 ~/matrix/inputs/molecule.gjf
matrix-submit g16 ~/matrix/inputs/molecule.gjf
matrix-submit molpro ~/matrix/inputs/molecule.com
matrix-submit orca ~/matrix/inputs/molecule.inp
```

Monitor jobs with:

```
matrix-status
matrix-tail 20260628-101010-gdv32-molecule
matrix-tail ~/matrix/logs/20260628-101010-gdv32-molecule.log
```

`matrix-submit` is a background wrapper based on ordinary Unix process management. It is not a queueing system. It is appropriate for this private machine, but it does not implement resource accounting, priorities or node scheduling. Each job writes `metadata.txt`; the fields `log`, `native_output` and `stdout` distinguish the stable log link, the scientific program output in the work directory and any launcher stdout/stderr stream.

6 Copying Files

Copy inputs from the local workstation to the remote input directory:

```
scp molecule.gjf enzo@oracle:~/matrix/inputs/
scp molecule.com enzo@oracle:~/matrix/inputs/
scp molecule.inp enzo@oracle:~/matrix/inputs/
```

Submit remotely:

```
ssh enzo@oracle 'matrix-submit gdv32 ~/matrix/inputs/molecule.gjf'
ssh enzo@oracle 'matrix-submit molpro ~/matrix/inputs/molecule.com'
```

After the run, copy back the files needed by MATRIX adapters. The safest target is the `native_output` path recorded in `metadata.txt`; for Gaussian this is normally a `.log`, for Molpro a `.out` and for ORCA a `.out`.

```
scp enzo@oracle:~/matrix/jobs/JOB_NAME/molecule.log .
scp enzo@oracle:~/matrix/jobs/JOB_NAME/molecule.fchk .
scp enzo@oracle:~/matrix/jobs/JOB_NAME/molecule.out .
```

Use the actual job directory name printed by `matrix-submit` or listed by `matrix-status`. Keep large scratch files remote unless they are needed for a specific adapter or viewer.

7 Returning Results To MATRIX

The completed QM output must be normalized before GF/PED, MORPHEUS, VPT2/VCI, DVR, spectroscopy or GUI tools consume it. Typical local commands after copying files back are:

```
matrix gaussian promote-fchk molecule.fchk molecule.xyzin
matrix gaussian promote-log-hessian molecule.log molecule.xyzin
matrix gaussian promote-rovib molecule.log molecule.xyzin
matrix gaussian promote-electronic molecule.log molecule.xyzin
matrix molpro promote-output molecule.out molecule.xyzin
```

For Gaussian ReadAllGIC optimization checks, keep the coordinate definition and the completed Gaussian result separate:

- the frozen MATRIX **#GIC** section is the authoritative coordinate source;
- the Gaussian input is only an external representation of those GICs;
- the Gaussian log supplies the final optimized geometry, Hessian and normal modes;
- GF/PED must rebuild B rows from the frozen MATRIX GIC definition on the final log geometry.

This avoids accidentally trusting a hand-edited or Gaussian-modified input block as the scientific coordinate definition.

8 Troubleshooting

Problem	Checks
Gaussian command not found	Run <code>setgau gdv32</code> , then <code>command -v gdv</code> . For standard Gaussian run <code>setgau g16</code> , then <code>command -v g16</code> .
Wrong Gaussian executable	Use only <code>gdv32</code> or <code>g16</code> . Do not use older GDV aliases.
Molpro unavailable	Run <code>module load molpro/2025.3</code> and <code>molpro -version</code> .
ORCA unavailable	Run <code>module load orca/6.1.1</code> and <code>command -v orca</code> .
No live output	Use <code>matrix-tail JOB_NAME</code> or inspect the log in <code>~/matrix/logs</code> .
Possible scratch/disk issue	Check <code>df -h ~/local/scratch</code> and the program log.
MATRIX tool cannot use a result	Promote the QM output first and verify that the required <code>xyzin</code> section is present with <code>matrix validate</code> or the GUI readiness panel.

9 Operational Policy

- Keep `oracle` as an execution backend, not a parallel MATRIX development tree.
- Keep one adapter per external format. Gaussian, Molpro and ORCA outputs must be normalized before scientific tools consume them.
- Use `gdv32` or `g16`; do not revive older GDV variants.
- Use Molpro 2025.3 through `module load molpro/2025.3`.
- Use ORCA through the configured ORCA module.
- Keep credentials outside the repository and outside documentation.
- Keep job provenance: input file, MATRIX version, remote command, program version, log path and promoted `xyzin` sections.